



Hi ERNIan, ask me a question.



Could you solve this with python? The unusual data (your puzzle input) consists of many reports, one report per line. Each report is a list of numbers called levels that are separated by spaces. For example:

7 6 4 2 1 1 2 7 8 9 9 7 6 2 1 1 3 2 4 5 8 6 4 4 1 1 3 6 7 9 This example data contains six reports each containing five levels.

The engineers are trying to figure out which reports are safe. The Red-Nosed reactor safety systems can only tolerate levels that are either gradually increasing or gradually decreasing. So, a report only counts as safe if both of the following are true:

The levels are either all increasing or all decreasing. Any two adjacent levels differ by at least one and at most three. In the example above, the reports can be found safe or unsafe by checking those rules:

7 6 4 2 1: Safe because the levels are all decreasing by 1 or 2. 1 2 7 8 9: Unsafe because 2 7 is an increase of 5. 9 7 6 2 1: Unsafe because 6 2 is a decrease of 4. 1 3 2 4 5: Unsafe because 1 3 is increasing but 3 2 is decreasing. 8 6 4 4 1: Unsafe because 4 4 is neither an increase or a decrease. 1 3 6 7 9: Safe because the levels are all increasing by 1, 2, or 3. So, in this example, 2 reports are safe.

Analyze the unusual data from the engineers. How many reports are safe?



Response time: 27.35 s

Yes, I can help you solve this problem using Python. Below is a Python script that reads the input data from a file (let's assume it's named `input.txt`), processes each report, and counts the number of safe reports according to the rules you've described.

First, let's outline the steps we need to follow:

1. **Read the input data:** Each line in the input file represents a report with levels separated by spaces.

2. **Process each report:** For each report, we'll:
 - Parse the levels into a list of integers.
 - Compute the differences between adjacent levels.
 - Check if all differences are either all positive (increasing) or all negative (decreasing).
 - Ensure that each difference is between 1 and 3 (inclusive) in absolute value.
3. **Count the safe reports:** Increment a counter for each report that meets the safety criteria.

Here's the Python code to accomplish this:

```
def is_safe_report(levels):
    """
    Determines if a report is safe based on the provided rules.
    """
    diffs = []
    for i in range(len(levels) - 1):
        diff = levels[i + 1] - levels[i]

        # Difference must be at least 1 and at most 3 in absolute value
        if abs(diff) < 1 or abs(diff) > 3:
            return False

        # Difference cannot be zero (levels cannot be equal)
        if diff == 0:
            return False

        diffs.append(diff)

    # All differences must have the same sign (all increasing or all decreasing)
    diff_signs = [diff > 0 for diff in diffs]
    if all(diff_signs) or not any(diff_signs):
        return True
    else:
        return False

def count_safe_reports(filename):
    """
    Reads the reports from the given file and counts how many are safe.
    """
    safe_reports_count = 0
    with open(filename, 'r') as file:
        for line in file:
            # Skip empty lines
            if not line.strip():
                continue

            levels = list(map(int, line.strip().split()))
            if is_safe_report(levels):
                safe_reports_count += 1
```

```
    return safe_reports_count

# Main execution
if __name__ == "__main__":
    input_file = 'input.txt' # Replace with the path to your input file
    total_safe_reports = count_safe_reports(input_file)
    print("Number of safe reports:", total_safe_reports)
```

Explanation:

- The `is_safe_report` function checks if a single report is safe by:
 - Calculating the differences between adjacent levels.
 - Checking if each difference is between 1 and 3 in absolute value.
 - Ensuring that the differences are either all positive or all negative (indicating strictly increasing or decreasing levels).
- The `count_safe_reports` function reads each line from the input file, parses it into levels, and uses `is_safe_report` to determine if it's safe.
- The `main` block sets the input file name, calls `count_safe_reports`, and prints the result.

Usage:

1. Save the code above to a file, e.g., `count_safe_reports.py`.
2. Make sure your input data is in a file named `input.txt` in the same directory as the script, or adjust the `input_file` variable to point to your input file.
3. Run the script using the command:

```
python count_safe_reports.py
```

4. The script will output the number of safe reports.

Example with Provided Data:

Given your example data, let's see how the script works.

Given `input.txt`:

```
7 6 4 2 1
1 2 7 8 9
9 7 6 2 1
1 3 2 4 5
8 6 4 4 1
1 3 6 7 9
```

Running the script will output:

Number of safe reports: 2

This matches your example where two reports are safe:

- 7 6 4 2 1
- 1 3 6 7 9

Note: Since you might have a different input file, make sure to use the correct file name and path. The script will process any number of reports in your input file according to the rules you've specified.

Your message

